

MORE PROGRAMS OR MORE ROLLS? SEPARATING COVERAGE FROM SPECIALIZATION IN LLM HARNESSES

Anonymous authors

Paper under double-blind review

ABSTRACT

Should an inference budget buy more programs or more executions of one program? We study this choice by separating answer coverage, repeatable performance, and usable selection in generated LLM harnesses. On 386 MATH-500 tasks, we compare eight generated programs plus a baseline with nine byte-identical baseline slots, each executed three times. Identical programs already yield 2.16 percentage points of repeat-averaged oracle headroom. Generated programs exhibit much stronger residual score-pattern correlation (0.789 versus 0.006), but this structure is dominated by persistent losses across 100 tasks; persistent wins occur on one. The frozen selector gains 0.00 percentage points, and both arms reach 98.70% oracle coverage at 27 executions. Held-out repeats leave stable complementarity unresolved. Supporting BIRD traces show how implementation, activation, and output validity shape executed behavior. In the tested panel, repeatable differences primarily reveal where programs lose, while additional baseline executions match full-budget coverage.

1 INTRODUCTION

Should a fixed inference budget buy more programs or more executions of one program? Automated agent design makes the first option increasingly accessible: a builder writes LLM harnesses that arrange prompts, tools, and control flow around a frozen solver (Hu et al., 2024; Nie et al., 2026; Zhang et al., 2026b). A population of such programs promises a division of labor, with different members serving different tasks. The practical value of that promise depends on whether the programs offer advantages that persist and can guide a choice.

The first sign of such an advantage is often additional coverage: one program answers a question that another misses. Yet the same pattern can arise from a single program. Two independent executions with success probability 0.5 cover a task with probability 0.75. Pass@ k captures this opportunity; self-consistency and sampling-and-voting ensembles turn multiple attempts into an aggregated answer (Chen et al., 2021; Wang et al., 2022; Li et al., 2024). An oracle that sees all outcomes can therefore reward a population even when every member has identical code. With only a few repeats, selecting the largest estimated success rate can continue to reward lucky outcomes.

We study how this initial coverage signal develops into useful task specialization. Three questions guide the analysis. Does the population supply additional correct answers? Do its task-wise advantages recur in a fresh execution? Can a rule choose a helpful program before execution, and how does the resulting coverage compare with additional attempts from a fixed program? Each question moves one step closer to a decision about where to spend the inference budget.

Our central experiment changes program identity while holding the number of members and repeats fixed. Eight generated harnesses plus single-call are compared with nine byte-identical baseline slots, each run three times on the same 386 MATH-500 tasks. Two repeats identify task-wise choices; the third tests whether those choices transfer. A selector frozen on development data then makes choices from question features, while an execution-matched replay compares the answers available at increasing budgets. The full population of 35 generated programs supplies the initial single-run coverage picture. Supporting BIRD traces follow program differences into execution, revealing how implementation and activation shape the behavior behind a score.

1. Coverage	2. Repeatability	3. Selection and replay
Repeat-mean oracle headroom	Residual score correlation	Frozen selector gain
Generated: 2.33 pp	Generated: 0.789	0.00 pp
Same code: 2.16 pp	Same code: 0.006	Oracle coverage, 27 executions
Identical programs also produce positive headroom.	Persistent losses: 100 tasks	Generated: 98.70%
	Persistent wins: 1 task	Same code: 98.70%

Figure 1: **Positive headroom and strong repeatability coexist with zero selector gain and equal full-budget oracle coverage.** All panels use the same 386 MATH tasks, eight generated programs plus single-call, and nine identical single-call slots, with three fresh executions per member. Persistent losses and wins are relative to single-call in all three repeats. Selector gain is relative to the development-selected fixed baseline. Replay compares the generated panel with same-code executions using post-execution correctness at matched harness execution counts.

The results tell a consistent story. Identical programs yield positive oracle headroom. Generated programs do have substantially more repeatable score patterns, but persistent losses span 100 tasks, while persistent wins occur on one. This structure maps weaknesses more clearly than useful task advantages: the clone-adjusted held-out gain is -0.26 pp, the frozen selector gains 0.00 pp, and both arms reach 98.70% oracle coverage at 27 executions.

The study contributes a same-code reference for interpreting oracle headroom, a repeated-execution analysis that separates persistent losses from wins, and a joint view of frozen selection and execution-matched coverage. Together, they make the population’s value concrete: which answers it adds, which differences persist, and which advantages inform a usable choice.

2 RELATED WORK

Automated agent and harness design. Executable programs provide a flexible space for adapting LLM behavior. ADAS searches this space directly (Hu et al., 2024); Self-Harness uses failure traces to propose edits and regression tests to retain them (Zhang et al., 2026b); TTHe adapts the harness during evaluation from unlabeled traces (Nie et al., 2026). Related systems improve task adaptation and validation allocation (Zhang et al., 2026a; Miyai et al., 2026), or expose harness behavior through verification, diagnosis, localization, and policy evolution (Xu et al., 2026; Chen et al., 2026a; Lin et al., 2026; Luo et al., 2026; Wang et al., 2026; Chen et al., 2026b). El et al. (2025) report low diversity in task-level score patterns among the automatically designed agents they study. We examine program populations through both execution traces and repeated outcomes: traces reveal which mechanisms execute, and same-code repeats measure coverage from sampling. This perspective connects program equivalence and mutation testing (Budd & Angluin, 1982; Papadakis et al., 2019) with performance attribution in generated harness populations.

Repeated sampling and ensembles. Multiple executions can improve answer coverage even with a fixed program. Pass@ k measures success within k samples; self-consistency and identical-agent ensembles aggregate attempts (Chen et al., 2021; Wang et al., 2022; Li et al., 2024), while nominally temperature-0 calls can also vary (Ouyang et al., 2023). These gains provide the sampling reference in our execution-matched comparison. A second distinction concerns when selection occurs. Post-execution verifiers rank candidate solutions (Kwok et al., 2026); ensemble selection chooses a model combination on validation data and averages predictions at inference (Caruana et al., 2004). Our frozen selector chooses a program before execution, while oracle replay measures coverage across recorded outputs.

Selection and resource allocation. Turning a population into a useful choice connects harness design to algorithm selection and portfolios (Rice, 1976; Xu et al., 2008; Kotthoff, 2014). HELIX uses complementary sibling trajectories to expand verified coverage and supply data for model improve-

ment (Fan & Huang, 2026). STS compares six reasoning paradigms and trains an embedding-based router to recover part of their oracle gap (Zhou et al., 2026). Harness sensitivity across model–problem pairs (Gupta et al., 2026) and evaluated models (Cho, 2026), search routing (Kim, 2026), and preference-trained model routing (Ong et al., 2024) further motivate task-dependent choices. GRASP selects edits to a bounded skill library under regression constraints (Moll et al., 2026). We hold the target model fixed and measure oracle coverage, repeat-stable task advantages, and realized selector gain in the same program population. The distinction between quantitative and qualitative interaction (Gail & Simon, 1985) clarifies why varying effect sizes can coexist with an unchanged best action.

3 PROBLEM FORMULATION

We distinguish the answers a population provides, the task advantages that persist across executions, and the gains a selector realizes. Let P contain baseline H_0 , and let $Y(h, x, r) \in \{0, 1\}$ denote member h ’s correctness on task x in repeat r . Accuracies use %; gains and headroom use percentage points (pp).

Coverage: which additional answers are available? A one-run oracle selects a member after seeing correctness. Its headroom over the best fixed member in the recorded matrix is

$$H(P) = \frac{1}{n} \sum_{i=1}^n \max_{h \in P} Y(h, x_i, r) - \max_{h \in P} \frac{1}{n} \sum_{i=1}^n Y(h, x_i, r). \quad (1)$$

This quantity captures available correct answers, including those obtained through repeated sampling. The repeat-averaged plugin $\hat{H}$ replaces Y with each member–task mean. With few repeats, maximizing those means can select favorable estimation errors, which motivates the same-code control.

Stable complementarity: which member is best for each task? Write $p_h(x) = \Pr\{Y(h, x, r) = 1\}$ under a fixed execution condition. The corresponding population headroom is $H_{\text{stable}} = \mathbb{E}_x \max_h p_h(x) - \max_h \mathbb{E}_x p_h(x)$. For two members with $\Delta(x) = p_1(x) - p_0(x)$,

$$H_{\text{stable}} = \frac{1}{2} (\mathbb{E}|\Delta(x)| - |\mathbb{E}\Delta(x)|). \quad (2)$$

Positive complementarity requires task-wise crossover: the better member changes with the task. If one member is better everywhere, headroom is zero even when its advantage varies in size. Identical programs under the same condition have equal $p_h(x)$, giving a zero-complementarity reference. This definition concerns population probabilities; the repeat diagnostic below tests choices learned from finite executions.

Usable selection: which advantage can guide a decision? A selector π_Z chooses from task features $Z(x)$ before execution. Its usefulness depends on whether those features predict the relevant advantage. We evaluate this decision separately from choices learned from earlier outcomes on the same task, which test repeat generalization. An execution-matched oracle replay then compares the coverage offered by a portfolio with additional fixed-program executions. Its budget counts harness executions; each execution may contain multiple model calls and tokens.

4 EXPERIMENTAL DESIGN

4.1 HARNESS POPULATIONS AND EVALUATION SETUP

We hold the target solver, GLM-5.3-Flash, fixed and vary the Python programs that call it. A *builder* writes each harness, the *solver* answers its model calls, and an external *judge* scores the final output. Gold answers are available only to the judge. Every harness implements `solve(question)` and calls `self.llm`; BIRD also provides schema and SQL feedback through `self.execute`. The single-call baseline makes one temperature-0 call. Generated programs can change prompts, decoding, sample count, and control flow. Table 1 maps the resulting populations to the questions they address; model details appear in Appendix A.3.

Study	Programs	Evaluation	Question answered
MATH exploration	35 generated + single-call	386 tasks; one run	Is there error coverage?
MATH repeat control	8 generated + single-call, versus 9 clones	Same 386 tasks; 3 fresh repeats	Do choices transfer across repeats?
BIRD discovery	12 generated + single-call + <code>react</code>	60 tasks; one run	What mechanisms execute?
BIRD admission	6 builders $\times$ 3 seeds $\times$ 4 arms	1,169 for primary contrast; 400 for all arms	Does screening change coverage?

Table 1: **Four studies connect coverage, repeatability, and execution.** One run means one execution per member–task pair. The repeated MATH panel is selected from the exploratory population. BIRD supplies separate execution and admission evidence, with effects reported by study.

MATH populations and panel selection. Six builders—GLM-5.3, Qwen3.8-Max, DeepSeek-V4-Pro, Kimi-K3, MiniMax-M3, and ERNIE-5.0-Thinking-Preview—each receive eight slots under one seed. A slot is an independent opportunity to improve on a single greedy call: the builder chooses the strategy, with no cross-slot diversity constraint. Each slot allows up to three attempts at temperature 0.7 and 32,768 output tokens, using the same prompt without feedback from earlier attempts. Generation stops at the first program that passes parsing, import, interface, and nonempty-answer checks on two development questions. The 48 slots yield 35 frozen programs (builder instructions: Appendix A.1).

The repeat panel samples eight of these programs. A historical mean-call ceiling of 3.05 leaves 21 eligible members; a fixed salted hash selects one per builder and two at-large seats. The resulting low-call panel includes draft–check–revise, agreement checking, and sample voting (Appendix A.2). Adding single-call gives nine members. This panel and nine identical baseline slots each receive three fresh, cache-disabled repeats, with decoding fixed by the source.

MATH-500 (Lightman et al., 2023) supplies 100 development problems and 386 evaluation problems in the common analysis set. We form this set after collection by requiring complete observations across both repeat arms and the single-run population, excluding 14 problems with incomplete repeat observations (Appendix B). All primary MATH comparisons therefore use the same task identities and fully observed scoring cells. The judge extracts the final answer, preferring the requested #### line, and compares numeric values when both parse or normalized LaTeX otherwise (Appendix A.1).

BIRD discovery and admission. BIRD (Li et al., 2023) requires SQL for a question and database. In the discovery study, TTHE’s proposer uses a GLM-5.3-Flash builder to produce 12 candidates that pass import and smoke-execution checks. These programs and two supplied baselines are evaluated on 60 questions. The admission study crosses free versus assigned strategies with ungated versus conformance-gated admission: six builders, three seeds, and eight slots per arm, with up to three attempts per slot (temperature 0.7; 16,384 output tokens). Assigned strategies include execution repair, three-sample voting, and schema linking. All arms require a valid interface and nonempty SQL; gated arms also check mechanism conformance and exclude some structures.

The admission split uses 365 questions from two databases for development and 1,169 from nine databases for evaluation, with a pre-drawn 400-question core shared by all four arms. An 18-item smoke test is a development-exposure exception. The official set-of-rows scorer compares predicted and gold query results, with a 30-second timeout and failures scored zero. Discovery uses an earlier stringified-row proxy with a row cap. Admission uses shared response caching, whereas the MATH repeats disable caching. Full split, scoring, and bundled gating specifications appear in Appendices E.1 and F.

4.2 SAME-CODE CONTROLS AND REPEAT VALIDATION

A same-code reference. The controlled panel contains nine distinct members; its reference contains nine byte-identical single-call slots. Each member or slot receives three executions per task. This design preserves population size and sampling opportunities while changing program identity.

Learning a choice on two repeats, testing it on the third. Each fold uses two discovery repeats to select a member for every task and a single best fixed member. The third repeat measures their accuracy difference. Averaging the three folds gives G_{real} and G_{clone} . We compare paired task-level gains on the common complete tasks through

$$D = G_{\text{real}} - G_{\text{clone}}. \quad (3)$$

A positive difference means that choices learned from past outcomes transfer better for distinct programs than for identical ones. Both the task-wise choice and the fixed comparator are determined entirely by the discovery repeats.

Analysis set and repeatability. All primary estimates use the same 386 tasks, with 10,422 scored cells per arm. The original incomplete-task fraction is 3.50%, below the acquisition gate’s 10% ceiling. The acquisition protocol preceded collection; the repeat diagnostic and common-complete-set analysis were developed afterward. The recovery snapshot preserves the programs, solver, tasks, and three-repeat design (Appendix B).

We also measure residual score-pattern correlation across repeats after removing additive member and task effects. This exploratory measure describes repeatability, while G describes held-out selection gain. Known-truth simulations characterize the diagnostic’s sensitivity. Appendices B.1 and C specify these calculations, tie handling, uncertainty estimates, and decision rules.

4.3 FROZEN SELECTION AND EXECUTION-MATCHED REPLAY

Selecting a program from the question. We train π_Z on the 100 development tasks using TF-IDF question features and training-fold member accuracies. Five-fold task-level cross-validation keeps each task’s repeats together. At inference, the five fold-model probability vectors are averaged; the policy selects the top member, falling back to single-call when the top-two probability gap is below 0.15. Ties follow fixed member-ID order. The complete policy is frozen before evaluation. Comparators include the development-selected fixed member and one randomly drawn fixed member.

Allocating executions to a portfolio. An offline oracle replay compares a development-ranked portfolio with same-code executions at budgets $b \in \{3, 9, 15, 27\}$. Each included member contributes three repeats. For clones, we compute exact mean coverage over uniform size- b subsets of the 27 recorded executions. Both arms use post-execution correctness to measure available answers. The recorded selector choices and portfolio order stay fixed for the recovery analysis. Replay uses tasks complete in both arms; its budget unit is a harness execution, with model-call and token use determined by the program. Appendix D gives the policy, replay formula, and accounting details.

5 RESULTS

5.1 THE POPULATION COVERS MORE TASKS THAN ANY SINGLE MEMBER

The generated population covers more tasks than its best fixed member. Across 386 analysis tasks, the union of 35 admitted generated members and single-call reaches 99.22% oracle coverage, compared with 95.08% for single-call. Every generated member is individually worse (69.69–92.49%). All 13,896 of 13,896 single-execution cells are observed, yielding 4.15 pp of additional coverage available to an oracle after execution.

This coverage comes with substantial variation in scored outcomes: 29 of the 36 correctness vectors are unique, mean pairwise disagreement is 10.94% (maximum 33.16%), and the union repairs 84.21% of baseline errors. The next question is how much of this variation survives repetition. We evaluate a selected program panel against same-code controls on the same 386 tasks, with a fresh baseline mean of 95.85%. The repeat-based D measures the panel’s held-out selection gain relative to clones; the full-population coverage and panel estimates refer to their respective populations and executions (Appendix A.1).

5.2 IDENTICAL PROGRAMS ALSO PRODUCE POSITIVE ORACLE ESTIMATES

Identical programs produce a 2.16 pp repeat-averaged oracle estimate, compared with 2.33 pp for the generated panel (Table 2). The nine clone slots implement the same program, so their population-level stable complementarity is zero by construction. Both estimates use the same 386 tasks with complete observations. Thus, even after averaging three repeats, a positive oracle estimate alone does not identify specialization.

Quantity	Generated + single-call	Same-code clones
Complete-task oracle estimate $\hat{H}$	2.33 pp	2.16 pp
Complete tasks for repeat-based G	386	386
Held-out-repeat mean gain G	0.10 pp	0.35 pp
Residual score-pattern correlation	0.789	0.006
Paired clone-adjusted mean: $D = -0.26$ pp on 386 tasks		
Analysis-set completeness: 100% in both arms (10,422 cells each)		

Table 2: **Coverage, repeatability, and selection gain separate.** All quantities use the same 386 tasks. Both arms are fully observed on this analysis set. D is the paired difference of the unrounded cross-fitted gains; rounding may affect the displayed difference.

5.3 REPEATABLE SCORE PATTERNS ARE DOMINATED BY LOSSES

Generated programs have more repeatable scored-outcome patterns than clones. Their mean residual correlation is 0.789, compared with 0.006 for clones; the paired-bootstrap difference is 0.783 (95% interval [0.69, 0.87]). This exploratory analysis removes additive member and task effects from the score-advantage matrix, then correlates each held-out repeat with the mean of the other two.

Persistent differences overwhelmingly favor single-call. All-three-repeat losses span 100 tasks (570 member-task pairs), whereas all-three-repeat wins occur on one task shared by all eight generated members. That apparent repair is sensitive to answer-format extraction (Appendix B.1). This directional asymmetry explains how strong score repeatability can coexist with few observed wins.

Held-out selection provides no clear positive evidence beyond clones. On the fully observed analysis tasks, the clone-adjusted mean is $D = -0.26$ pp, with 95% interval $[-1.45, 0.95]$ pp and randomization $p = 0.8127$. The interval includes zero and the positive-support conditions are not met, giving INSUFFICIENT EVIDENCE rather than an equivalence conclusion. Excluding the three tasks containing conflicting scores gives $D = -0.61$ pp on 383 paired tasks and the same decision (Appendix B).

Three repeats have limited power for crossovers. Known-truth calibration shows that three repeats can miss substantial task-wise crossovers. A two-specialist setting with true $H_{\text{stable}} = 17.5$ pp has 0% power at $R = 3$, about 0.7% at $R = 5$, and 100% at $R = 10, 20$. Across the tested 100-cell grid, the worst observed null false-support rate is 2%; smaller cyclic crossovers remain largely undetected even at $R = 20$ (Appendix Table 7). These setting-specific results explain why useful crossovers remain unresolved at the current repeat count. We next evaluate the tested pre-execution decision rule directly.

5.4 THE TESTED SELECTOR ADDS NO GAIN AND FULL-BUDGET COVERAGE TIES

The frozen selector matches the development-selected baseline. The tested feature policy yields 0.00 pp gain: after applying its confidence-based fallback, it selects single-call on all 386 evaluation tasks. Keeping these choices fixed and updating only the recorded outcomes gives 95.85% accuracy over three repeats, identical to the development-selected fixed member. This result evaluates the complete decision rule; the reported choices do not distinguish initial classifier preferences from fallback decisions. Appendix D gives its construction and the same-set comparator results.

At the full execution budget, the oracle coverages tie. At $b = 27$ executions, the development-ranked portfolio and same-code controls both reach 98.70% oracle coverage (Table 3). The compar-

ison uses the 386 tasks complete in both arms and averages clone coverage over uniformly selected executions. At $b = 9$ and 15, the portfolio-minus-clone differences are +0.08 pp and +0.02 pp. At $b = 3$, the portfolio contains only single-call, so the +0.20 pp difference reflects execution variation between identical programs.

Harness executions per task b	3	9	15	27
Panel members, including single-call	1	3	5	9
Panel oracle coverage (%)	98.45	98.70	98.70	98.70
Clone oracle coverage (%)	98.24	98.63	98.69	98.70
Panel minus clones (pp)	+0.20	+0.08	+0.02	+0.00

Differences are computed before rounding the displayed coverage values.

Table 3: **Full-budget oracle coverage ties in the recorded executions.** Each included member contributes three repeats. Clone coverage is the exact mean over uniform subsets of its 27 recorded executions, paired on complete tasks. Both arms use post-execution correctness at matched harness execution counts.

Together, the selector and replay results separate two operational questions: the tested pre-execution policy yields zero gain, while post-execution oracle coverage ties at the full recorded budget. That budget matches harness executions; model-call counts and token costs vary by program.

5.5 BIRD TRACES LOCATE BREAKS BETWEEN CODE AND EXECUTION

Source variation can disappear before it produces a valid final answer. The supporting BIRD study follows this process directly in independent program populations, using its task-specific judging and protocols. Here a mechanism is an implemented prompt transformation, tool use, or control-flow operation. This execution view complements the MATH controls by examining what the programs actually do.

Different source code often produces identical scored outcomes. All 12 discovery candidates pass a code-difference check, yet 21/66 generated-candidate pairs have identical correctness vectors (36/91 including baselines). Source similarity has essentially no rank association with correctness disagreement ($\rho = -0.010$). This agreement on 60 tasks motivates examining the traces (Appendix Figure 2).

Traces distinguish absent, inactive, and broken mechanisms. T1: described but absent. Six of twelve candidates describe retrieval or self-checking but make a single model call without an implemented retrieval or self-checking step. This classification covers all twelve candidates; the five-harness trace audit supplies worked examples.

T2: present but untriggered. On all ten audited `react` tasks, the first SQL executes successfully, so the repair branch never fires and final SQL equals single-call. A successful execution can therefore leave a semantically wrong answer unchanged.

T3: executed but broken. One candidate runs multi-turn repair but returns prose as final SQL, losing tasks that the baseline answers correctly. The three cases locate distinct checks: implementation, branch activation, and valid output. Appendix E provides worked traces and hand-written controls, with their development-exposure limitations.

The separate BIRD admission study finds a mean headroom contrast of -0.16 pp for forced/gated versus free/ungated generation. Its gate combines structural exclusion and conformance verification (Appendix F). Together, these BIRD observations make the execution path concrete: a source-level difference must be implemented, activated, and carried through to a valid answer. Repeatable task advantages and selection value measure what that execution subsequently delivers.

6 DISCUSSION AND LIMITATIONS

From a population of programs to a useful choice. The three measurements describe successive parts of the same story. Coverage records the answers a population makes available. Repeats reveal

which differences persist. Selection tests whether those differences help a decision made before execution. In the generated panel, the strongest repeatable structure is a map of losses: persistent losses span 100 tasks, while persistent wins occur on one. The frozen selector provides a direct practical result, 0.00 pp gain, alongside the equal full-budget oracle coverage. For harness design, the relevant target is a repeatable task advantage that a selection rule can recognize. BIRD adds an execution-level view: implementation, activation, and a valid final output are successive requirements for a proposed mechanism to affect behavior.

Scope. The MATH study uses a low-call panel, one solver, and three repeats. The repeat diagnostic and complete-case reanalysis were developed after collection; the recovery stage preserved programs, tasks, solver, and repeat count. The 14 excluded tasks are harder than the 386 included tasks (Appendix B). The held-out comparison leaves stable complementarity unresolved at three repeats, and recorded score patterns include answer-format effects. Selector results concern the frozen policy; replay matches harness executions, with model calls and tokens varying by program. BIRD uses its own populations and scoring conditions.

7 CONCLUSION

A harness population can find more answers, exhibit repeatable differences, and still provide little value to a selector. The tested MATH panel brings these properties into the same comparison: identical programs produce positive oracle headroom, generated score patterns repeat mainly through persistent losses, and the frozen selector gains 0.00 pp. At 27 executions, both arms reach 98.70% oracle coverage. These findings make the resource-allocation question concrete. The value of more programs lies in task advantages that persist, guide selection, and improve on additional executions of a fixed program.

ETHICS STATEMENT

All experiments use public BIRD and MATH-500 benchmark data and generated program artifacts; no human participants, private records, or personally identifiable information are collected. Model calls are reported as part of the experimental provenance, and the paper does not release credentials or provider responses that would expose private data. The results distinguish recorded answer coverage from the performance of a selector operating before execution.

REPRODUCIBILITY STATEMENT

The manuscript specifies the populations, task splits, execution conditions, scoring rules, statistical procedures, and sensitivity analyses used for the reported results. The appendices provide additional protocol details, calibration results, and worked examples. The review materials present the methodological and empirical evidence through these descriptions, tables, and examples.

AI USE STATEMENT

Generative AI tools assisted with research questions, experimental and statistical design, counterexample analysis, code development, model-API workflows, data analysis, figures, and manuscript preparation. Versioned artifacts record the analyses and revisions; automated tests, local runtime checks, and same-model subagents supported mechanical verification. The human authors set the research direction and retain responsibility for scientific decisions, verification, the final content, and submission.

REFERENCES

- Timothy A. Budd and Dana Angluin. Two notions of correctness and their relation to testing. *Acta Informatica*, 18(1):31–45, 1982. doi: 10.1007/bf00625279.
- Rich Caruana, Alexandru Niculescu-Mizil, Geoff Crew, and Alex Ksikes. Ensemble selection from libraries of models. In *Proceedings of the 21st International Conference on Machine Learning*, 2004. doi: 10.1145/1015330.1015432.

- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. doi: 10.48550/arXiv.2107.03374. URL <https://arxiv.org/abs/2107.03374>.
- Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*, 2026a. doi: 10.48550/arXiv.2606.06324. URL <https://arxiv.org/abs/2606.06324>.
- Mingju Chen, Can Lv, Guibin Zhang, Heng Chang, and Shiji Zhou. HarnessForge: Joint harness and policy evolution for adaptive agent systems. *arXiv preprint arXiv:2606.01779*, 2026b. doi: 10.48550/arXiv.2606.01779. URL <https://arxiv.org/abs/2606.01779>.
- Yong-eun Cho. It’s not the capability: Harness sensitivity is non-monotone across LLM agent tiers. *arXiv preprint arXiv:2605.26731*, 2026. doi: 10.48550/arXiv.2605.26731. URL <https://arxiv.org/abs/2605.26731>.
- Batu El, Mert Yuksekgonul, and James Zou. Inefficiencies of meta agents for agent design. *arXiv preprint arXiv:2510.06711*, 2025. doi: 10.48550/arXiv.2510.06711. URL <https://arxiv.org/abs/2510.06711>.
- Tianyu Fan and Chao Huang. HELIX: Model-harness co-evolution for recursive self-improvement. *arXiv preprint arXiv:2608.13951*, 2026. doi: 10.48550/arXiv.2608.13951. URL <https://arxiv.org/abs/2608.13951>.
- Mitchell H. Gail and Richard Simon. Testing for qualitative interactions between treatment effects and patient subsets. *Biometrics*, 41(2):361–372, 1985. doi: 10.2307/2530862.
- Akshat Gupta, Jermaine Lei, Alexander Lu, Gopala Anumanchipalli, and Leshem Choshen. Automated discovery has no universally superior harness. *arXiv preprint arXiv:2607.18235*, 2026. doi: 10.48550/arXiv.2607.18235. URL <https://arxiv.org/abs/2607.18235>.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024. doi: 10.48550/arXiv.2408.08435. URL <https://arxiv.org/abs/2408.08435>.
- Minho Kim. When should LLMs search? counterfactual supervision for search routing. *arXiv preprint arXiv:2607.05752*, 2026. doi: 10.48550/arXiv.2607.05752. URL <https://arxiv.org/abs/2607.05752>.
- Lars Kotthoff. Algorithm selection for combinatorial search problems: A survey. *AI Magazine*, 35(3):48–60, 2014. doi: 10.1609/aimag.v35i3.2460.
- Jacky Kwok, Shulu Li, Pranav Atreya, Yuejiang Liu, Yixing Jiang, Chelsea Finn, Marco Pavone, Ion Stoica, and Azalia Mirhoseini. LLM-as-a-verifier: A general-purpose verification framework. *arXiv preprint arXiv:2607.05391*, 2026. doi: 10.48550/arXiv.2607.05391. URL <https://arxiv.org/abs/2607.05391>.
- Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can LLM already serve as a database interface? a BIG bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, volume 36, pp. 42330–42357, 2023. doi: 10.52202/075280-1835. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/83fc8fab1710363050bbd1d4b8cc0021-Abstract-Datasets_and_Benchmarks.html.
- Junyou Li, Qin Zhang, Yangbin Yu, Qiang Fu, and Deheng Ye. More agents is all you need. *arXiv preprint arXiv:2402.05120*, 2024. doi: 10.48550/arXiv.2402.05120. URL <https://arxiv.org/abs/2402.05120>.

- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023. doi: 10.48550/arXiv.2305.20050. URL <https://arxiv.org/abs/2305.20050>.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*, 2026. doi: 10.48550/arXiv.2604.25850. URL <https://arxiv.org/abs/2604.25850>.
- Xiaotian Luo, Dizhan Xue, Fengxingyu Wang, Chuanrui Hu, and Yafeng Deng. HarnessBank: Semantic gene-bank search with gated verification for agent-harness self-evolution. *arXiv preprint arXiv:2607.13683*, 2026. doi: 10.48550/arXiv.2607.13683. URL <https://arxiv.org/abs/2607.13683>.
- Atsuyuki Miyai, Kiyoharu Aizawa, and Toshihiko Yamasaki. Task-CoEvolve: Efficient harness optimization via adaptive validation task selection. *arXiv preprint arXiv:2608.20169*, 2026. doi: 10.48550/arXiv.2608.20169. URL <https://arxiv.org/abs/2608.20169>.
- Johannes Moll, Jean-Philippe Corbeil, Jiazhen Pan, Martin Hadamitzky, Daniel Rueckert, Lisa Adams, and Keno Bressem. GRASP: Gated regression-aware skill proposer for self-improving LLM agents. *arXiv preprint arXiv:2605.29668*, 2026. doi: 10.48550/arXiv.2605.29668. URL <https://arxiv.org/abs/2605.29668>.
- Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Yike Guo, Xinmei Tian, and Bo Han. TTHe: Test-time harness evolution. *arXiv preprint arXiv:2607.08124*, 2026. doi: 10.48550/arXiv.2607.08124. URL <https://arxiv.org/abs/2607.08124>.
- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route llms with preference data. *arXiv preprint arXiv:2406.18665*, 2024. doi: 10.48550/arXiv.2406.18665. URL <https://arxiv.org/abs/2406.18665>.
- Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. An empirical study of the non-determinism of ChatGPT in code generation. *arXiv preprint arXiv:2308.02828*, 2023. doi: 10.48550/arXiv.2308.02828. URL <https://arxiv.org/abs/2308.02828>.
- Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: an analysis and survey. In *Advances in Computers*, volume 112, pp. 275–378. Elsevier, 2019. doi: 10.1016/bs.adcom.2018.03.015.
- John R. Rice. The algorithm selection problem. In *Advances in Computers*, volume 15, pp. 65–118. Elsevier, 1976. doi: 10.1016/s0065-2458(08)60520-3.
- Ruhan Wang, Yucheng Shi, Zongxia Li, Zhongzhi Li, Yue Yu, Junyao Yang, Kishan Panaganti, Haitao Mi, Dongruo Zhou, and Leoweiliang. Harness handbook: Making evolving agent harnesses readable, navigable, and editable. *arXiv preprint arXiv:2607.13285*, 2026. doi: 10.48550/arXiv.2607.13285. URL <https://arxiv.org/abs/2607.13285>.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022. doi: 10.48550/arXiv.2203.11171. URL <https://arxiv.org/abs/2203.11171>.
- Jinghan Xu, Yikai Zhang, Aili Chen, Weiyuan Li, Jiaqing Liang, and Deqing Yang. Verify smarter, evolve further: Efficient harness evolution through behavior-aware verification. *arXiv preprint arXiv:2608.27311*, 2026. doi: 10.48550/arXiv.2608.27311. URL <https://arxiv.org/abs/2608.27311>.
- Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32:565–606, 2008. doi: 10.1613/jair.2490.

Guibin Zhang, Leo Lu, Fangzhou Xie, Kang Zhu, Junhao Wang, Zhifei Xie, Zhaochen Yu, Zihang Liu, Zhongxiang Sun, Qiankun Li, Yue Liao, Heng Chang, Xiaobin Hu, Qibing Ren, Wangchunshu Zhou, Chuanrui Hu, Yafeng Deng, and Shuicheng Yan. JIT-Agent: Scaling harness intelligence via just-in-time harness evolution. *arXiv preprint arXiv:2608.25593*, 2026a. doi: 10.48550/arXiv.2608.25593. URL <https://arxiv.org/abs/2608.25593>.

Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026b. doi: 10.48550/arXiv.2606.09498. URL <https://arxiv.org/abs/2606.09498>.

Heng Zhou, Zelin Tan, Zhemeng Zhang, Yutao Fan, Yibing Lin, Li Kang, Xiufeng Song, Rui Li, Songtao Huang, Ao Yu, Yuchen Fan, Yanxu Chen, Kaixin Xu, Xiaohong Liu, Yiran Qin, Philip Torr, Chen Zhang, and Zhenfei Yin. Select-then-solve: Paradigm routing as inference-time optimization for LLM agents. *arXiv preprint arXiv:2604.06753*, 2026. doi: 10.48550/arXiv.2604.06753. URL <https://arxiv.org/abs/2604.06753>.

APPENDIX GUIDE

The appendices document program generation, repeated execution, selection, calibration, and the supporting BIRD studies.

A Harness generation, program inventory, and data splits.

B Same-code controls and repeated-execution protocol.

C Statistical analysis, calibration, and sensitivity.

D Frozen selector and execution-matched replay.

E BIRD execution traces and supporting analyses.

F BIRD admission-policy study.

A HARNESS GENERATION, PROGRAM INVENTORY, AND DATA SPLITS

This appendix describes the MATH population, splits, frozen panel, and model versions. BIRD protocols appear in Appendices E and F.

A.1 MATH POPULATION, COMMON ANALYSIS SET, AND COVERAGE

The single-run study supplies coverage measurements and programs for the repeat panel, applying BIRD’s free, ungated generation procedure to competition mathematics with a domain-specific comparator.

Protocol and analysis set. The builder instruction, summarized rather than quoted, requests a Python `solve(question)` harness that improves on one greedy call through `self.llm`, with freely chosen prompting, decoding, sampling, and control flow and a final `####` answer. Gold answers remain external. Slots receive no assigned mechanism or diversity target. Attempts reuse the instruction without prior code or performance feedback; admission requires a valid interface and nonempty output.

Six builders each receive eight slots under one seed, with at most three attempts per slot. Parsing, import, interface, and nonempty answers are checked on two development questions. The target is GLM-5.3-Flash, with member-specific decoding and temperature 0 for single-call. The pre-generation split reserves 100 development problems. Single-run, repeat, and policy analyses share 386 fully observed tasks; the post-collection completeness rule excludes 14 problems (Appendix B). Development tasks and the selector stay fixed.

The judge compares parsed numbers, including fractions and decimals, or otherwise normalized LaTeX. Normalization removes units, base subscripts, and degree symbols, retains text-command content, and maps matrices to tuples. Gold answers prefixed with `####` pass 386/386 self-comparisons. Single-run baseline accuracy on this set is 95.08%.

The gate admits 35/48 slots. GLM-5.3 and Qwen3.8-Max exhaust the 32k-token generation budget before emitting code in most slots, leaving their styles under-represented in this fixed-budget population.

Results on the common complete set. All 36 members, including single-call, have one score per task: 13,896/13,896 cells over 386 tasks. Mean pairwise disagreement is 10.94% (maximum 33.16%), with 29/36 unique outcome vectors (28/35 generated) and 2.22% identical pairs. The generated union repairs 84.21% of baseline errors; all generated members fail on 0.82% of baseline-correct tasks. Oracle coverage of 99.22% exceeds the best fixed member single-call (95.08%) by 4.15 pp. Generated accuracies span 69.69%–92.49%.

The repeat panel compares eight generated members plus single-call with nine same-code slots on identical tasks. Its held-out diagnostic concerns this panel; single-run headroom concerns the full population. Both the archived judge and the versioned endpoint-sensitive judge reproduce all 13,896 included scores.

Scope. These measurements describe one seed and one target on the completeness-selected MATH set; BIRD uses the separate protocols below.

A.2 SELECTED MATH PROGRAM INVENTORY

Table 4 describes the nine frozen members selected for the repeat study. H1–H8 are neutral labels for generated members, all constructed under seed 0. The listed operations describe what each program implements, not whether every branch fires or improves correctness. The mean-call eligibility ceiling is not a per-execution maximum, and one logical call may request several samples.

Frozen member	Source-defined execution path
single-call	One temperature-0 solution with the final-answer format.
H1 (DeepSeek)	Three temperature-0.4 solutions; normalized-answer vote; a greedy fall-back when no answer or a tie remains.
H2 (ERNIE)	Draft a solution, then ask the solver to verify/correct the extracted answer; return the checked answer.
H3 (GLM)	Up to five differently prompted temperature-0 solutions; stop when three agree; adjudicate unresolved disagreement, with format fallbacks.
H4 (Kimi)	Request five temperature-0.7 samples in one call; extract answers and vote.
H5 (Kimi)	Initial solution, verification, and an alternative solution; vote, with an additional resolution call if no majority emerges.
H6 (MiniMax)	Draft, critique, and an additional tie-breaking call on disagreement; parse failures use an earlier available answer.
H7 (MiniMax)	Request five temperature-0.7 samples in one call; normalize and vote, resolving ties by first occurrence.
H8 (Qwen)	Draft plus an independent solve-and-audit call; compare answers and request adjudication on conflict, with a format-repair path.

Table 4: **What was selected for the repeat study.** All calls use the same frozen solver; verification and adjudication are solver prompts, not access to benchmark labels. Six seats are builder-stratified; H5 (Kimi) and H7 (MiniMax) are the two at-large seats.

A.3 MODELS AND VERSIONS

Table 5 gives the exact API model identifiers. All roles use one commercial OpenAI-compatible aggregator. Target identity is fixed; decoding follows each harness, with temperature 0 for the single-call baseline. Phase-II generation uses temperature 0.7 and a 16,384-token budget; Phase-I conditions appear in Appendix E.1. Phase I uses a proxy execution judge, Phase II the official BIRD scorer (30 s timeout) with a secondary legacy-judge audit, and MATH-500 the answer comparator in Appendix A.1. Reported accuracy uses no LLM judge.

Role	Model (exact version string)	Used in
Frozen target solver	GLM-5.3-Flash (baseline temperature 0)	MATH / BIRD
BIRD discovery builder	GLM-5.3-Flash	App. E.1
MATH / BIRD admission builders	GLM-5.3, Qwen3.8-Max, DeepSeek-V4-Pro, Kimi-K3, MiniMax-M3, ERNIE-5.0-Thinking-Preview	MATH / BIRD
Judge	proxy / official BIRD execution scorer; MATH answer comparator	see text

Table 5: Model inventory for the reported benchmark studies. The target is held frozen within each phase; Phase-II builders come from six independent labs.

B SAME-CODE CONTROLS AND REPEATED-EXECUTION PROTOCOL

All MATH primary estimates use the same 386 fully observed tasks. The protocol below specifies the panel, inclusion rule, and repeat analysis.

Panel and inclusion. Programs, solver, judge, development split, and three repeat identities are fixed. A salted hash selects one low-call member per builder plus two at-large members and single-call; the control contains nine identical baseline slots. The September 26, 2026, 05:15 UTC snapshot combines generated continuation 10 and clone continuation 20. After outcome inspection, we retained tasks with a recorded binary score in every required single-run and repeat cell. These 386 task identities define the common reanalysis set.

Completeness and exclusions. The single-run matrix contains $36 \times 386 = 13,896$ observed cells and each repeat arm $9 \times 3 \times 386 = 10,422$, giving 100% completeness for all three matrices. Policy evaluation and oracle replay use the same task identities.

The 14 excluded problems contain 61 unscored generated cells, distributed as DeepSeek 5, ERNIE 4, GLM 9, Kimi 30, MiniMax 9, and Qwen 4; baseline and clone records are complete. Their pre-subsetting fraction, 3.50%, is below the frozen 10% gate. Excluded tasks are harder and longer, with lower baseline accuracy (Table 6). Unknown scores and all incurred acquisition costs remain in the source records.

Observed characteristic	Included (386)	Excluded (14)
Baseline repeat-mean accuracy (%)	95.85	71.43
Mean benchmark difficulty level	3.47	4.29
Level-5 tasks (%)	26.42	64.29
Mean question length (characters)	198.75	284.71

Table 6: **Observed characteristics of included and excluded tasks.** Excluded tasks have higher benchmark difficulty, longer questions, and lower baseline accuracy.

Provenance and conflicts. Manifests are checked on 13 fields covering execution conditions, programs, task/split hashes, panel draw, and schedule; snapshots, matrices, and task lists are hash-bound. Completed scores replace unknowns, with first-completed precedence on disagreement. Five generated cell identities conflict, including three on three retained tasks; clones have none. Collector and concurrency changes are logged.

Estimates and sensitivity. Recorded binary scores give plug-in headroom of 2.33 pp (generated) and 2.16 pp (clone), and $D = -0.26$ pp (95% interval $[-1.45, 0.95]$ pp; INSUFFICIENT EVIDENCE). Removing the three conflict-affected tasks from both arms and reselecting on the remaining 383 tasks gives $D = -0.61$ pp (interval $[-1.96, 0.64]$ pp), with the same decision.

B.1 EXPLORATORY REPEATABILITY AND SCORE SENSITIVITY

For non-reference members, define $A_r(h, x) = Y(h, x, r) - Y(h_0, x, r)$, where h_0 is single-call or clone-c1. Within each repeat, subtract row and column means and add the grand mean. Correlate each held-out residual matrix with the residual of the other two repeats' mean, then average the three correlations. A paired task bootstrap uses 2,000 shared draws. Correlations are 0.789 ([0.75, 0.83]) for generated members and 0.006 ([-0.07, 0.08]) for clones, with difference 0.783 ([0.69, 0.87]). This post-collection diagnostic measures repeatable score patterns after removing additive effects.

Losses in all three repeats span 570 member-task pairs on 100 tasks. Wins in all three comprise eight pairs on one task, shared by all eight generated programs. That task is format-sensitive: the gold answer is 12, baseline outputs end with ##### 12th grade, and a generated member returns 12th grade. The extractor retains the former's complete ##### payload and scores it zero, but uses numeric fallback for the latter and scores it one. Recorded scores are retained; the repeated repair here arises from answer formatting.

C STATISTICAL ANALYSIS, CALIBRATION, AND SENSITIVITY

Empirical diagnostics use 386 complete MATH tasks. Synthetic calibration uses the sample sizes specified below.

C.1 HELD-OUT-REPEAT STATISTIC AND DECISION RULE

In each of three folds, two discovery repeats select a member for each task and a single best fixed member by mean discovery accuracy. The third repeat scores the task-wise mapping against that fixed member. Averaging task-level gains over folds gives G ; the paired difference between arms is $D = G_{\text{real}} - G_{\text{clone}}$ (§4.2). The repeat-averaged oracle plug-in $\hat{H}$ is descriptive and does not determine the decision.

Discovery scores lie on $\{0, \frac{1}{2}, 1\}$, so ties are common. Selection uses iid $U(0, 10^{-9})$ jitter to break member ties symmetrically, averaging ten frozen tie realizations for the empirical report. Each calibration dataset uses one tie realization. For the empirical randomization reference, member labels are permuted independently within each task and each repeat in each arm. Both selections, both gains, and D are recomputed for each of 10,000 draws, with the +1 Monte Carlo correction. A paired task bootstrap uses 2,000 draws with a fixed random seed and the same resampled task identities in both arms to obtain the 95% interval.

Positive support requires: at most 10% incomplete tasks in either arm or their intersection before subsetting; at least 30 paired tasks; randomization $p < 0.05$; positive bootstrap lower bounds for D and G_{real} ; and $D > 1.0$ pp. Repeats must have matched conditions and verified independent execution provenance. The current decision is INSUFFICIENT EVIDENCE, rather than an equivalence finding.

Analysis specification. The acquisition protocol and initial stopping rule preceded collection; the v3 diagnostic, recovery, and completeness-selected reanalysis are post-collection amendments. Estimates use the September 26 snapshot; inclusion, exclusions, and conflict sensitivity are specified in Appendix B.

C.2 POPULATION TARGET AND LIMITS OF THE DIAGNOSTIC

For member success probabilities $q_h(x)$, stable complementarity is

$$H_{\text{stable}} = \mathbb{E}_x \max_h q_h(x) - \max_h \mathbb{E}_x q_h(x).$$

It is zero when some fixed member is expectation-best on almost every task. A selector's population gain θ^* relative to the *population-best fixed* member satisfies $\theta^* \leq H_{\text{stable}}$, because its success probability on each task is at most $\max_h q_h(x)$.

Empirical G uses a fixed member selected from finite discovery data. Clone subtraction provides a sampling reference, while D and its interval remain distinct from H_{stable} : a near-zero D does not bound the population target near zero.

The randomization reference is exact under equal-ability member-label exchangeability. For the broader global-dominance null, error control is empirical over the calibration grid.

Finite-repeat oracle estimates. For repeat means $\hat{q}_h(x)$, Jensen’s inequality gives $\mathbb{E} \max_h \hat{q}_h(x) \geq \max_h \mathbb{E} \hat{q}_h(x)$ on each task. Headroom subtracts a second maximum, so this inequality does not imply upward bias of the difference in general. In a known-truth equal-ability simulation with $q = 0.9$, $M = 9$, $T = 400$, and $R = 3$, the mean plug-in is 8.69 pp over 300 datasets despite $H_{\text{stable}} = 0$. The empirical same-code arm likewise gives 2.16 pp. These examples show that positive plug-in headroom is insufficient evidence of specialization.

C.3 KNOWN-TRUTH CALIBRATION AND STATISTICAL POWER

Binary tensors are sampled from specified member–task probability matrices. Null settings include equal ability, shared task difficulty, global dominance, the 0.5/0.9 dominance configuration, heterogeneous dominance, and matched sticky within-task latent states. Alternatives include crossovers over latent task types and difficulty-related partial crossover. The grid spans $M \in \{3, 9, 20\}$, $T \in \{100, 400, 1000\}$, $R \in \{3, 5, 10, 20\}$, complete data, 5% MCAR, and 3% hardness-related missingness.

Table 7 reports $M = 9$, $T = 400$, using 300 Monte Carlo datasets per cell (150 in sweeps) and 300 randomizations per dataset. Across the 100-cell grid, the largest observed null false-support rate is 2%. A two-specialist alternative with $H_{\text{stable}} = 17.5$ pp has power 0% at $R = 3$, approximately 0.7% at $R = 5$, and 100% at $R = 10, 20$; smaller cyclic crossovers remain largely undetected at $R = 20$. These setting-specific results show why three repeats leave stable complementarity unresolved.

regime	$R=3$		$R=10$		$R=20$		abst.
	plug	v3	plug	v3	plug	v3	$R=3$
equal $q=0.9^\dagger$	100	0	100	0	100	1	100
equal + difficulty [†]	100	0	100	1	100	1	100
0.5/0.9 dominance [†]	100	0	100	0	13	0	100
dominance [†]	100	0	100	0	100	0	100
cross $H \approx 0.9$ pp	100	1	100	0	100	0	99
cross $H \approx 1.8$ pp	100	1	100	1	100	0	99
cross $H \approx 4.2$ pp	100	0	100	0	100	0	100
two strong specialists, $H=17.5$ pp	100	0	100	100	100	100	100

[†]Null ($H=0$): false-support %, nominal 5%; else power %. $M=9$, $T=400$, $B=300$ at $R=3$ (150 sweeps), 300 randomizations.

Appendix C describes the M , T , R and missingness settings and summarizes the wider calibration grid.

Table 7: **Three repeats can miss even a strong positive case.** At $M = 9$, $T = 400$, null rows report false-support % and alternative rows report power %; the final column gives $R = 3$ abstention. The clone-calibrated rule (v3) rarely supports nulls, but has little sensitivity to the tested small and moderate crossovers.

D FROZEN SELECTOR AND EXECUTION-MATCHED REPLAY

The selector is fitted on 100 development tasks. Its recorded choices and development ranking are evaluated on the common 386 tasks.

D.1 FROZEN FEATURE SELECTOR

The reported selector π_Z combines five multinomial-logistic fold models fitted using the 100 development tasks. Its inputs are TF-IDF question features (at most 300) and per-member development accuracies computed within the respective training folds. All repeats of a task stay together in the five-fold task partition. The score summary for member h on development task x is the available-repeat mean,

$$\bar{Y}_h(x) = \frac{1}{|\mathcal{R}_{hx}|} \sum_{r \in \mathcal{R}_{hx}} Y(h, x, r),$$

where $\mathcal{R}_{hx}$ contains its observed development repeats. The five fitted models are combined by fold-model averaging. If $p_k(h \mid x)$ is the probability assigned by fold model k , the inference rule uses $\bar{p}(h \mid x) = \frac{1}{5} \sum_{k=1}^5 p_k(h \mid x)$. Let $h_{(1)}$ and $h_{(2)}$ be the members with the two highest averaged probabilities, breaking ties by member ID. The final frozen choice is

$$\pi_Z(x) = \begin{cases} \text{single-call,} & \bar{p}(h_{(1)} \mid x) - \bar{p}(h_{(2)} \mid x) < 0.15, \\ h_{(1)}, & \text{otherwise.} \end{cases}$$

Thus the five-fold step supplies the averaged prediction used at inference, and the threshold is applied after averaging. The policy table evaluates these final choices, including the baseline fallback. It reports repeat-mean accuracy, with harness executions as the unit of evaluation; a selected harness can make multiple model calls.

Comparators are single-call, the development-selected fixed member, one fixed member drawn once with a fixed seed, and the development-selected member with baseline fallback on missing cells.

Policy	Eval acc. (%)	S vs dev-fixed (pp) [95% CI]
single-call (fixed)	95.85	0.00 [0.00, 0.00]
dev-fixed	95.85	—
One randomly drawn fixed member	86.01	-9.84 [-13.04, -6.91]
Dev-fixed + baseline fallback	95.85	0.00 [0.00, 0.00]
π_Z (frozen selector)	95.85	0.00 [0.00, 0.00]

Table 8: **Frozen comparators on the common 386-task MATH analysis set.** Every selected member has all three scores on every task. Accuracy is repeat-mean correctness; paired differences are against dev-fixed with 2,000 task-bootstrap draws. The feature policy, dev-fixed, and fallback policy all retain their recorded single-call choices. The randomly drawn fixed member is `gsm_kimi_s0_g3`. Recorded policies are evaluated on fresh executions of these tasks.

D.2 SELECTION GAIN AND THE SCOPE OF BUDGET UTILITY

For the implemented selector, $Z(x)$ consists only of question features and training-fold member accuracies. Its gain over the development-selected fixed member is

$$S(\pi_Z) = \mathbb{E}_x [\bar{Y}_{\pi_Z(x)}(x) - \bar{Y}_{h_{\text{dev-fixed}}}(x)],$$

where $\bar{Y}_h(x)$ now averages the three evaluation repeats. We use S for policy gain and reserve D for the real-minus-clone repeat diagnostic. Mechanism labels are not inputs to this feature selector.

Replay budget b counts *harness executions*, each potentially containing multiple model calls; coverage uses a post-execution oracle.

D.3 EXECUTION-MATCHED REPLAY AND COLLECTION ACCOUNTING

Each arm contains nine members/slots with three repeats. Replay uses the common 386 tasks and frozen development ranking (§5.4), taking all three repeats of the first $b/3$ members for $b \in \{3, 9, 15, 27\}$. With f incorrect clone executions among 27, expected coverage of a uniform size- b subset is $1 - \binom{f}{b} / \binom{27}{b}$, where $\binom{f}{b} = 0$ for $b > f$. Table 3 compares this exact expectation with generated-panel coverage at matched execution counts.

Acquisition ledgers retain all attempts and known usage, including excluded tasks and recovery; outcome tables use the common complete set. Calls and tokens follow each program and acquisition record.

E BIRD EXECUTION TRACES AND SUPPORTING ANALYSES

Phase I studies BIRD discovery; Phase II studies admission. This appendix details the discovery programs and their execution paths.

E.1 DISCOVERY POPULATIONS AND CONDITIONS

Population and judging. Discovery evaluates twelve TTHE-proposer candidates from GLM-5.3-Flash, plus single-call and `react`, on 60 tasks (§5.5). Admission checks import and smoke execution. The baseline flips on three of 60 tasks between discovery runs. The set-of-rows proxy judge agrees with the official BIRD scorer on 0.97–0.99 of outcomes in two matrices with retained SQL, with no proxy-correct/official-incorrect cases.

Development exposure. Hand-written controls were designed after trace inspection, and 24/151 evaluation items overlap protocol development. Their headroom is descriptive. Equal generation budgets cannot be reconstructed from the early records, which lack generation-tool and per-attempt logs.

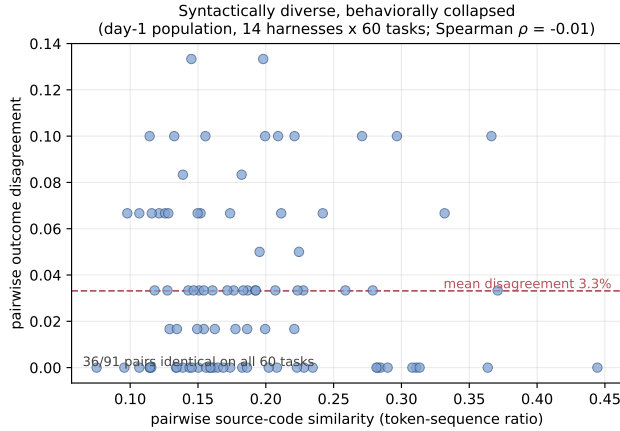


Figure 2: **Source differences need not change observed correctness.** BIRD discovery: 14 harnesses, 60 tasks, one execution each. Across 91 pairs including baselines, token-sequence source similarity has $\rho = -0.010$ rank association with correctness disagreement; 36/91 pairs have identical correctness vectors. This is an outcome comparison on the observed tasks, not a test of program equivalence.

E.2 DISCOVERY MEASUREMENTS AND HAND-WRITTEN CONTROLS

All candidates pass the code-difference check, but 21/66 generated pairs share correctness vectors (36/91 with baselines). Source similarity and correctness disagreement have $\rho = -0.010$ across 91 pairs (Figure 2). On these 60 tasks, mean disagreement is 3.3% overall and 3.8% among generated candidates, union repair is 7.7%, and single-run headroom is 3.3 pp. Traces explain how distinct programs produce these closely aligned outcomes.

E.3 TRACES DISTINGUISH ABSENCE, NON-ACTIVATION, AND FAILURE

T1: described but absent. Six of twelve candidates claim retrieval or self-checking but make one model call without implementing either step. **T2: present but untriggered.** On all ten audited `react` tasks, the first SQL executes without error, leaving the repair branch idle and final SQL identical to single-call; execution success can coexist with semantic error. **T3: executed but broken.** One of twelve candidates executes multi-turn repair but returns prose as SQL, losing baseline-correct tasks.

The three cases separate implementation, branch activation, and output validity. Four hand-written controls with distinct paths yield 6.67 pp headroom on 60 tasks and 7.28 pp on 151 items. These post-inspection controls lack matched repeats and clones; their 24/151 development overlap is documented in Appendix E.1. Worked traces follow in Appendix E.4.

E.4 WORKED EXAMPLES FROM THE TRACE AUDIT

The six-of-twelve T1 count covers the full discovery population. The separate trace audit follows five harnesses—single-call, *react*, single-call, timeout-retry, and multi-turn repair—on ten tasks (5×10). Below, one task illustrates these paths. Quoted SQL is capped by the logger at 120 characters; truncation is marked [*trunc.*].

The task. BIRD dev-set question 352 (*card_games* database; question text verbatim, the typo is the dataset’s):

```
Calculate the percentage of the cards availabe in Chinese
Simplified.
```

Its BIRD hint:

```
'Chinese Simplified' is the language; percentage =
Divide(Sum(id where language = 'Chinese Simplified'),
Count(id)) *100
```

Gold SQL (executing the two on the released database yields 8.7734 and 8.7728 respectively — the official scorer separates them):

```
SELECT CAST(SUM(CASE WHEN T2.language = 'Chinese Simplified'
THEN 1 ELSE 0 END) AS REAL) * 100 / COUNT(T1.id) FROM cards
AS T1 INNER JOIN foreign_data AS T2 ON T1.uuid = T2.uuid
```

The single-call harness answers:

```
SELECT SUM(CASE WHEN language = 'Chinese Simplified'
THEN 1 ELSE 0 END) * 100.0 / COUNT(id) AS percentage FROM
foreign_data;
```

marked wrong by the official scorer: it counts directly over *foreign_data*, whereas the gold query counts after joining to *cards*; the hint does not spell out that join.

***react* (T2: untriggered repair).** The first prompt is byte-identical to single-call’s. SQL executes successfully, so the database-error retry does not fire. All 10 audited tasks follow this path, with SQL execution but 10/10 final queries byte-identical to single-call’s.

Single-call candidate (T1: absent repair). The docstring’s *conservative repair* specifies an output-token cap and answer-only block. Execution consists of one model call followed by SQL extraction. The cap is an environment default (2,048 versus 32,000 at runtime), has no harness control-flow implementation, and never binds on these short answers. The worked task returns the same wrong *foreign_data* query, modulo whitespace, with one call and zero SQL executions. Across all 10 tasks, SQL executions remain zero and 6 final queries are byte-identical to single-call’s. The remaining differences reflect a reworded prompt, without retrieval, self-check, or repair.

Timeout-retry candidate (untriggered watchdog). A daemon-thread watchdog wraps the coder call and issues one hedged duplicate after a 40 s stall. No stall occurs in the audit: SQL executions are zero and final SQL equals single-call’s on all 10 tasks.

Multi-turn repair candidate (T3: a broken multi-turn path). The one audited candidate implementing genuine execute-inspect-repair returns the model’s *prose* as final SQL. On a task single-call answered correctly (“State the alternative languages available for card named Annul numbered 29.”), its logged final SQL begins, verbatim:

```
with "only a single read-only SELECT is allowed". The
previous "query" was actually just prose text (the model
wrote rea[trunc.]
```

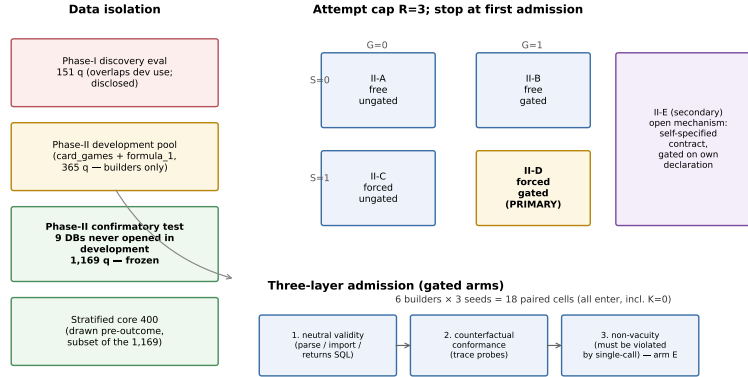
The repair re-prompt invites the failure: asked to fix a query that “was actually just prose”, the frozen target responds with more prose reasoning, no SQL block is found, and the reasoning text is submitted as the query. The item flips from single-call-correct to harness-wrong.

F BIRD ADMISSION-POLICY STUDY

The 2×2 study crosses six builders and three seeds; II-E is development-only. Each slot allows $R = 3$ attempts and stops at first admission without top-up. Two development databases are excluded. Evaluation uses 1,169 questions from nine databases and a pre-drawn 400-item core, with one disclosed 18-item smoke-test exposure. The official BIRD judge uses a 30 s timeout. Model-specific response caching uses cross-process locks after correction of a concurrency defect.

The primary D–A contrast pairs builder–seed cells with hierarchical bootstrap and permutation inference; core analyses use Holm-corrected factorial contrasts and K-matched sensitivity. Instrument fixes preceded confirmatory outcomes. Subsequent analysis corrections share baseline records and database/task resampling. Canonical records use first-write-wins deduplication; conflicts and operational deviations remain in the source audit. Generation costs vary across arms.

Phase-II confirmatory design: database-level isolation, 2×2 factorial + open-mechanism arm, three-layer admission



II-E: development-only; excluded from confirmatory comparisons.

Figure 3: **BIRD admission design.** Left: database-level contamination isolation. Middle: the 2×2 confirmatory factorial with a common attempt cap; the additional open-mechanism arm II-E belongs only to development. Right: the three-layer admission pipeline.

F.1 DESIGN AND PRIMARY CONTRASTS

The six-builder, three-seed study crosses strategy forcing and gating: II-A (free, ungated), II-B (free, gated), II-C (forced, ungated), and II-D (forced, gated). Forced strategies come from a fixed set; free builders select a declaration within the vocabulary in Appendix F.4. The evaluation uses 1,169 questions from nine databases, with the disclosed 18-item smoke-test exception, and the official BIRD scorer. Results below use the corrected reanalysis of the pre-outcome specification.

Metric	II-A	II-D
Candidate count	5.78	3.83
Oracle accuracy (%)	76.14	75.70
Best-fixed accuracy (%)	67.58	67.30
Headroom (pp)	8.56	8.40
D–A headroom (pp)	−0.16	
Unadjusted bootstrap 95% CI	[−0.83, +0.87]	
One-sided sign-flip p	0.647	

Table 9: **Corrected primary analysis.** Means over 18 builder–seed cells on 1,169 questions; single-call is included in both the oracle and best-fixed comparators.

The forced/gated minus free/ungated contrast is −0.16 pp in baseline-inclusive oracle headroom, with cell contrasts −4.36 to +2.74 pp. The implemented bundle does not improve observed mean headroom; Table 9 reports uncertainty.

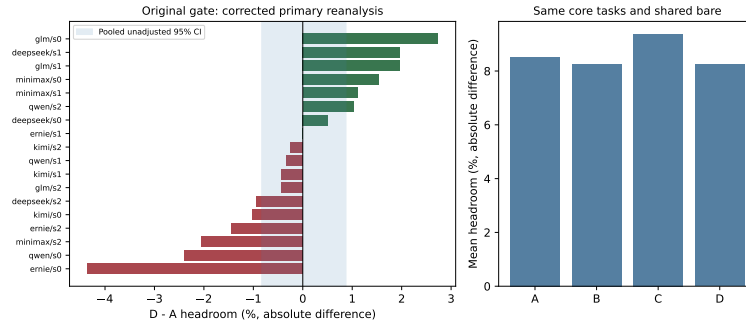


Figure 4: **Versioned corrected reanalysis.** Left: full-set D–A cell contrasts with the pooled 95% interval. Right: four-arm headroom on the shared core-400 tasks with a common single-call baseline. Headroom and contrast axes show percentage points.

On core-400, the gate contrast is −0.69 pp, with inconclusive strategy and interaction effects (Table 10). The fixed- K contrast is −0.31 pp. II-E remains development-only. The R2/R3 audits (Appendix F.3) place these single-run coverage measurements alongside 8.54% R2 flips, 14.5% of baseline tasks changing across three runs, and a 6.50 pp same-code oracle gap.

Contrast	Effect (pp)	95% CI (pp)	Raw p	Holm p
Gate	−0.69	[−1.56, +0.20]	0.026	0.079
Strategy	+0.42	[−0.32, +1.51]	0.263	0.428
Interaction	−0.87	[−2.83, +0.65]	0.214	0.428

Table 10: **Corrected core-400 factorial reanalysis.** Gate is $[(B-A) + (D-C)]/2$, strategy $[(C-A) + (D-B)]/2$, interaction $(D-C) - (B-A)$; raw and Holm-adjusted p -values.

F.2 SAME-CORE HEADROOM DECOMPOSITION

Table 11 separates coverage, best-fixed accuracy, and headroom using the same core tasks and baseline records in every arm.

Arm	K_{cand}	K_{total}	Single-call (%)	Best (%)	Oracle (%)	H (%)
A	5.78	6.78	63.25	66.00	74.51	8.51
B	6.11	7.11	63.25	66.12	74.39	8.26
C	5.61	6.61	63.25	65.64	75.01	9.38
D	3.83	4.83	63.25	65.65	73.90	8.25

Table 11: **Same-core headroom decomposition.** All arms use identical 400 tasks and identical single-call outcomes. K_{cand} excludes single-call and $K_{\text{total}} = K_{\text{cand}} + 1$; counts and accuracies are cell means. Headroom is oracle minus best-fixed accuracy, not realized routing gain.

F.3 SENSITIVITY, REPEAT AUDITS, AND RECORDED COST

K-matched and R2 replays. Replays use deterministic record ordering, shared database/within-database task draws, and seed resampling within fixed builders (10,000 replicates). K-matching uses the smaller candidate count in each A/B and C/D pair, exactly enumerates uniform subsets, retains single-call, and reselects best-fixed within each subset and task draw. It measures population-size sensitivity. R2, described below, retains all 18 cells, including two singleton A subsets; its D–A changes from +0.42 to +0.81 pp (change +0.39 pp, 95% CI [-0.95, +1.63]).

R2 covers 70 harnesses on 400 tasks, taking the first two callable admitted A/D candidates where available. Two cache-off acquisition groups remain separate despite their duplicated repeat label, with program identity checked against cached runs. Without matched single-call acquisitions, both R2 conditions exclude that baseline; singleton headroom is zero across the 18 retained pairs. Recorded flips are 2391/28,000 against cached outcomes (8.54%) and 1245/14,400 between D cache-off passes (8.65%). These describe observed instability: acquisition time and caching are confounded, and the rates are not standard errors or effect-size floors.

R3 single-call-only repeat audit. Three independent cache-off runs evaluate only the single-call baseline on core-400 with GLM-5.3-Flash at temperature 0. Correctness changes on 58/400 tasks (14.5%); pairwise flips are 39/400, 40/400, and 37/400 (9.75%, 10.0%, 9.25%). Treating runs as three pseudo-members, one execution each, gives official-scorer oracle coverage 72.25% and best-repeat accuracy 65.75%: a 6.50 pp same-code gap. This records baseline execution variability at temperature 0 and motivates the repeat controls used in the main study.

Sensitivity estimand	Estimate (%)	95% CI (%)
K-matched B–A headroom	−0.55	[−1.53, +0.63]
K-matched D–C headroom	−0.08	[−1.26, +0.80]
K-matched gate mean	−0.31	[−1.07, +0.40]
R2 subset D–A (cached)	+0.42	[−0.61, +1.48]
R2 subset D–A (cache-off)	+0.81	[−0.35, +1.84]
R2 change in subset D–A	+0.39	[−0.95, +1.63]

Table 12: Corrected sensitivities on core-400, 18 cells. CIs are unadjusted. K-matched comparisons always retain single-call; R2 comparisons exclude single-call and are conditional on the observed acquisition groups.

Recorded cost. The core records contain 259,304 logical solver calls across 153,600 candidate–task records (Table 13). These include cache-served requests and count logical rather than billed requests; token usage is unavailable. Generation attempts, admissions, and execution calls are reported separately. Population cost averages weight builder–seed cells equally, exclude single-call, and assign zero calls to zero-candidate populations.

Arm	Candidate-task records	Logged solver calls	Calls/task/population
A	41,600	67,258	9.34
B	44,000	64,399	8.94
C	40,400	75,604	10.50
D	27,600	52,043	7.23

Table 13: Descriptive canonical cost audit on core-400, 18 cells per arm. Logged logical calls are not billable requests or token counts. The last column averages candidate-population calls per task over cells.

F.4 ADMISSION SCOPE AND IMPLEMENTATION

Free-arm builders choose among repair, vote, twostage, and plain. Vote requires multiple samples and selection, including but not limited to three-way majority voting. Twostage consumes an artifact without a prescribed response interface. Plain declares no control-flow mechanism despite the prompt requesting a change. Thus strategy choice operates within this four-declaration vocabulary.

The evaluated gate combines structural screening with conformance verification; valid prompt-only transformations can fail the structural screen. Development checks found false rejections, request-attribution and response-interface defects, and unresolved judgments. Later revisions produced no replacement benchmark outcomes, so results describe the implemented bundle.

Generation interface. All 108 format-guard attempts across 36 forced-arm cells fail neutral validity: 71 logs report unterminated strings, 36 no Python fence, and one a provider timeout. Synthetic examples reproduce truncation in the archived non-greedy fence extractor. Full builder messages and exact historical extractor versions are unavailable, limiting attribution of these upstream validity failures.